

41052 - Advanced Algorithms
Programming Assignment 1 - Fast Fourier Transform

Tung Nguyen - 24666048

Contents

1	A little history	2
2	Relevant background knowledge	2
2.1	Discrete Fourier Transform	2
2.2	Fast Fourier Transform	3
2.2.1	Radix-2 Fast Fourier Transform	3
2.2.2	Optimising with Precomputed Twiddle	6
2.2.3	Mixed Radix Fast Fourier Transform	7
3	Implementation	7
4	The Empirical Study	7
5	Lesson learned	7
6	AI usage Declaration	7

1 A little history

The Discrete Fourier Transform wasn't invented by Joseph Fourier. It actually predated Fourier's work on the continuous Fourier Transform (formally published in 1822)! In 1759, Alexis Clairaut published an early discrete trigonometric formula while calculating celestial orbits. In 1805, Carl Friedrich Gauss refined this technique to track the paths of asteroids, effectively devising both the discrete Fourier formula and an early Fast Fourier Transform (FFT). However, his work remained largely overlooked until long after his death because it was recorded in unpublished Latin notes with unconventional notation. Since this formula handles discrete values with methods like the Fourier transform, later on it was named as the Discrete Fourier Transform.

The modern breakthrough occurred during the Cold War. As the United States and the Soviet Union negotiated nuclear test bans, verifying compliance required distinguishing clandestine underground nuclear tests from natural earthquakes. Processing the flood of discrete seismic sensor data was computationally impractical using standard methods, which scaled at $\mathcal{O}(N^2)$.

Physicist and defence adviser Richard Garwin recognised this bottleneck and raised it with statistician John Tukey from Princeton and Bell Labs. Tukey proposed a divide-and-conquer approach to accelerate the computation and worked with IBM mathematician James Cooley, who implemented and programmed the algorithm. Published in 1965, the Cooley–Tukey FFT slashed the computational cost to $\mathcal{O}(N \log N)$ (fundamentally, Gauss invented the same algorithm but his work went unnoticed). By that time, major countries had already created their nuclear devices and went ahead with their own testing.

Nowadays, the FFT has revolutionised digital signal processing, telecommunications and numerical computing worldwide and become one of the most important algorithms in our lives, as it is used every phone and personal device and so much more.

2 Relevant background knowledge

2.1 Discrete Fourier Transform

The Discrete Fourier Transform (DFT) is a function that receives an array input of complex numbers and returns a transformed array with the same amount of elements. The DFT can be defined as an algorithm.

Algorithm 1 Discrete Fourier Transform

Input: Array $x[0 \dots N - 1]$ of complex numbers, length N

Output: Transformed array $X[0 \dots N - 1]$

```
1: procedure DFT( $x, N$ )
2:   Allocate array  $X[0 \dots N - 1]$  initialised to 0
3:   for  $k \leftarrow 0$  to  $N - 1$  do
4:     for  $n \leftarrow 0$  to  $N - 1$  do
5:        $\theta \leftarrow \frac{2\pi}{N} \cdot k \cdot n$ 
6:        $W \leftarrow \cos \theta - i \sin \theta$  # The  $W$  is known as the twiddle factor
7:        $X[k] \leftarrow X[k] + x[n] \cdot W$ 
8:   return  $X$ 
```

Looking deeper into this algorithm, the first for loop (line 2) is going over every element of the input array of size N (line 6, with $X[k]$). The nested for loop (line 3) is also looping over size N to compute for each value of array X . Thus, the DFT has a time complexity of $\mathcal{O}(N^2)$.

The Inverse Discrete Fourier Transform (IDFT) is used for reversing a transformed array to its original array, essentially undoing the DFT. It can also be described as an algorithm.

Algorithm 2 Inverse Discrete Fourier Transform

Input: Transformed array $X = [X[0], X[1], \dots, X[N-1]]$ of size N

Output: Original array $x = [x[0], x[1], \dots, x[N-1]]$

```
1: procedure IDFT( $x, N$ )
2:   Allocate array  $x[0 \dots N-1]$  initialised to 0
3:   for  $n \leftarrow 0$  to  $N-1$  do
4:     for  $k \leftarrow 0$  to  $N-1$  do
5:        $\theta \leftarrow \frac{2\pi}{N} \cdot k \cdot n$ 
6:        $W \leftarrow \cos(\theta) + i \sin(\theta)$ 
7:        $x[n] \leftarrow x[n] + X[k] \cdot W$ 
8:    $x[n] \leftarrow \frac{1}{N} \cdot x[n]$ 
9:   return  $x$ 
```

Once again, due to the nested for loops (line 2 and 3), the IDFT has a complexity of $\mathcal{O}(N^2)$. Moreover, it is true that

$$\text{IDFT}(x) = \frac{1}{N} \cdot \text{conj} [\text{DFT}(\text{conj}(x))]. \quad (1)$$

In this property, $\text{conj}()$ is a function that turn a complex number into its complex conjugate (flipping the operator of the imaginary part). This operation is looping over N input of the array, hence it's complexity is $\mathcal{O}(N)$ and is bounded by the above mentioned $\mathcal{O}(N^2)$. Simply speaking, for all of the mentioned algorithm, the IFFT is just the FFT with different factors involved due to the $\text{conj}()$.

2.2 Fast Fourier Transform

The Fast Fourier Transform (FFT) is a class of algorithms that can compute the DFT and IDFT faster than $\mathcal{O}(N^2)$. In addition, as mentioned in equation 1, when computing for IDFT, a new complexity of $\mathcal{O}(N)$ is added. Since FFT essentially compute DFT, Inverse Fast Fourier Transform (IFFT) can be used to compute for IDFT. And all of the following algorithms' complexity is $\mathcal{O}(N \log_2 N)$, the $\text{conj}()$ complexity is also bounded.

2.2.1 Radix-2 Fast Fourier Transform

The most famous FFT algorithm is the Cooley-Tukey FFT (also known as the Radix-2 FFT) mentioned above. There are two ways to implement this algorithm: iterative and recursive. The core idea for this FFT is to apply the divide and conquer strategy to the input array.

Iterative Radix-2 FFT

The iterative version runs with the help of the Bit Reversal Permutation (BRP) algorithm. The BRP essentially loop over an array, for each index it flips the binary (e.g. $001 \mapsto 100, 110 \mapsto 011$), and move the current value to its new index. The reason for using the BRP will be discussed in Section 4.

Algorithm 3 Bit-Reversal Permutation

Input: Array $A[0 \dots N-1]$, length $N = 2^m$

Output: Array A permuted in-place into leaf order

```
1: procedure BIT-REVERSE-PERMUTATION( $A, N, m$ ) #  $m \leftarrow \log_2 N$ 
2:   for  $i \leftarrow 0$  to  $N-1$  do
3:      $rev \leftarrow 0$ 
4:      $temp \leftarrow i$ 
5:     for  $b \leftarrow 0$  to  $m-1$  do
6:        $bit \leftarrow temp \bmod 2$  # Extract binary split parity
7:        $rev \leftarrow 2 \cdot rev + bit$  # Shift accumulated bits left and append
8:        $temp \leftarrow \lfloor temp/2 \rfloor$  # Advance to next binary digit
9:     if  $i < rev$  then
10:       $swap(A[i], A[rev])$ 
```

Algorithm 4 Iterative Radix-2 FFT

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^m$

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N$ )
2:    $m \leftarrow \log_2 N$ 
3:   BIT-REVERSE-PERMUTATION( $X, N, m$ )
4:   for  $s \leftarrow 1$  to  $\log_2 N$  do
5:      $m \leftarrow 2^s$ 
6:      $W_m \leftarrow \exp(-i \frac{2\pi}{m})$ 
7:     for  $k \leftarrow 0$  to  $N-1$  step  $m$  do
8:        $W \leftarrow 1$ 
9:       for  $j \leftarrow 0$  to  $m/2-1$  do
10:         $t \leftarrow W \cdot W[k+j+m/2]$ 
11:         $u \leftarrow W[k+j]$ 
12:         $W[k+j] \leftarrow u+t$ 
13:         $W[k+j+m/2] \leftarrow u-t$ 
14:         $W \leftarrow W \cdot W_m$ 
15:   return  $A$ 
```

Looking deeper into the Iterative version of the Radix-2 FFT, the first part of the algorithm is spent running the BRP algorithm, which first cost $\mathcal{O}(N)$ looping over the entire array. And a nested for loop inside that cost $\mathcal{O}(\log_2 N)$. In total, the BRP algorithm cost $\mathcal{O}(N \log_2 N)$. Next, the algorithm loops (line 4) over $\log_2 N$ times, followed by a nested loop (line 7) of $\frac{N}{m}$ times (indicated by the **step** m), followed by another nested loop (line 9) of $\frac{m}{2}$ times. In total, this FFT algorithm cost $\mathcal{O}(N \log_2 N + \log_2 N \cdot \frac{N}{m} \cdot \frac{m}{2}) = \mathcal{O}(N \log_2 N)$.

Algorithm 5 Iterative Radix-2 IFFT

Input: Transformed array $X[0 \dots N-1]$ of complex numbers, length $N = 2^m$

Output: Original array $x[0 \dots N-1]$

```
1: procedure FFT( $x, N$ )
2:    $m \leftarrow \log_2 N$ 
3:   BIT-REVERSE-PERMUTATION( $X, N, m$ )
4:   for  $s \leftarrow 1$  to  $\log_2 N$  do
5:      $m \leftarrow 2^s$ 
6:      $W_m \leftarrow \exp(i \frac{2\pi}{m})$ 
7:     for  $k \leftarrow 0$  to  $N-1$  step  $m$  do
8:        $W \leftarrow 1$ 
9:       for  $j \leftarrow 0$  to  $m/2-1$  do
10:         $t \leftarrow W \cdot W[k+j+m/2]$ 
11:         $u \leftarrow W[k+j]$ 
12:         $W[k+j] \leftarrow u+t$ 
13:         $W[k+j+m/2] \leftarrow u-t$ 
14:         $W \leftarrow W \cdot W_m$ 
15:   return  $A$ 
```

Similarly, this IFFT algorithm cost $\mathcal{O}(N \log_2 N)$.

Recursive Radix-2 FFT

Algorithm 6 Recursive Radix-2 FFT

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^m$ **Output:** Transformed array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N$ )
2:   if  $N = 1$  then
3:     return  $x$ 
4:   Allocate array  $x_{\text{even}}$  of length  $N/2$ 
5:   Allocate array  $x_{\text{odd}}$  of length  $N/2$ 
6:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
7:      $x_{\text{even}}[k] \leftarrow x[2k]$ 
8:      $x_{\text{odd}}[k] \leftarrow x[2k + 1]$ 
9:    $E \leftarrow \text{FFT}(x_{\text{even}}, N/2)$  # Sub-transform of even elements
10:   $O \leftarrow \text{FFT}(x_{\text{odd}}, N/2)$  # Sub-transform of odd elements
11:  Allocate array  $X$  of length  $N$ 
12:  for  $k \leftarrow 0$  to  $N/2 - 1$  do
13:     $W \leftarrow \exp(-i \frac{2\pi}{N} \cdot k)$ 
14:     $t \leftarrow W \cdot O[k]$ 
15:     $X[k] \leftarrow E[k] + t$ 
16:     $X[k + N/2] \leftarrow E[k] - t$ 
17:  return  $X$ 
```

The Recursive version of the Radix-2 FFT first copy into two arrays x_{even} and x_{odd} (line 6 to 8), with each costing $\mathcal{O}(\frac{N}{2})$. Then, it recursively called with inputs of size $\frac{N}{2}$ two times (line 9 and 10), each time costing $T(\frac{N}{2})$ (since it's recursion, we cannot use Big $\mathcal{O}$ right away). Finally, the algorithm loop over $\frac{N}{2}$ times (line 12). Thus, the entire algorithm cost

$$T(N) = 2\mathcal{O}\left(\frac{N}{2}\right) + 2T\left(\frac{N}{2}\right) + \mathcal{O}\left(\frac{N}{2}\right) = \mathcal{O}(N) + 2T\left(\frac{N}{2}\right).$$

According to the Master Theorem, we have

$$T(N) = aT\left(\frac{N}{b}\right) + f(N).$$

It is clear that $f(n) = \mathcal{O}(N)$. Moreover, $a = 2, b = 2 \Rightarrow N^{\log_2 2} = N$. This matches with the case 2 of the Master Theorem. Therefore,

$$T(N) = \mathcal{O}(N \log_2 N).$$

Algorithm 7 Recursive Radix-2 IFFT

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^m$ **Output:** Transformed array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N$ )
2:   if  $N = 1$  then
3:     return  $x$ 
4:   Allocate array  $x_{\text{even}}$  of length  $N/2$ 
5:   Allocate array  $x_{\text{odd}}$  of length  $N/2$ 
6:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
7:      $x_{\text{even}}[k] \leftarrow x[2k]$ 
8:      $x_{\text{odd}}[k] \leftarrow x[2k + 1]$ 
9:    $E \leftarrow \text{FFT}(x_{\text{even}}, N/2)$  # Sub-transform of even elements
10:   $O \leftarrow \text{FFT}(x_{\text{odd}}, N/2)$  # Sub-transform of odd elements
11:  Allocate array  $X$  of length  $N$ 
12:  for  $k \leftarrow 0$  to  $N/2 - 1$  do
13:     $W \leftarrow \exp(i \frac{2\pi}{N} \cdot k)$ 
14:     $t \leftarrow W \cdot O[k]$ 
15:     $X[k] \leftarrow E[k] + t$ 
16:     $X[k + N/2] \leftarrow E[k] - t$ 
17:  return  $X$ 
```

Similarly, the IFFT version would also cost $\mathcal{O}(N \log_2 N)$.

2.2.2 Optimising with Precomputed Twiddle

Algorithm 8 Iterative Radix-2 FFT with Precomputed Twiddle Table

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^m$, precomputed table $W[0 \dots N/2-1]$

where $W[k] = e^{-i\frac{2\pi}{N}k}$

Output: Transformed array $X[0 \dots N-1]$

```

1: procedure PRECOMPUTE-TWIDDLES( $N$ )
2:   Allocate table  $W$  of size  $N/2$ 
3:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
4:      $W[k] \leftarrow \cos\left(\frac{2\pi k}{N}\right) - i \sin\left(\frac{2\pi k}{N}\right)$ 
5:   return  $W$ 

6: procedure FFT( $X, N, W$ )
7:   BIT-REVERSE-PERMUTATION( $A, N$ )
8:   for  $s \leftarrow 1$  to  $\log_2 N$  do                                     # Stage loop
9:      $m \leftarrow 2^s$                                                 # Sub-transform size
10:     $step \leftarrow N/m$                                              # Stride to sample twiddles from global table
11:    for  $k \leftarrow 0$  to  $N-1$  step  $m$  do                             # Block loop
12:      for  $j \leftarrow 0$  to  $m/2 - 1$  do                               # Butterfly loop
13:         $twiddle \leftarrow W[j \cdot step]$                             #  $\mathcal{O}(1)$  table lookup
14:         $t \leftarrow twiddle \cdot A[k + j + m/2]$ 
15:         $u \leftarrow A[k + j]$ 
16:         $A[k + j] \leftarrow u + t$ 
17:         $A[k + j + m/2] \leftarrow u - t$ 
18:    return  $A$ 

```

2.2.3 Mixed Radix Fast Fourier Transform

Algorithm 9 General Mixed-Radix Cooley–Tukey FFT (On-the-Fly Twiddles)

Input: Sequence $x[0 \dots N-1]$, factors N_1, N_2 such that $N = N_1 \times N_2$

Output: Transformed frequency array $X[0 \dots N-1]$

```

1: procedure MIXED-RADIX-FFT( $x, N_1, N_2$ )
2:   Allocate 2D matrices  $A[0 \dots N_1-1][0 \dots N_2-1]$  and  $B[0 \dots N_1-1][0 \dots N_2-1]$ 
3:   Allocate 1D array  $X[0 \dots N-1]$ 
4:    $N \leftarrow N_1 \cdot N_2$ 
                                     # Step 1: Map 1D input into  $N_1 \times N_2$  matrix

5:   for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
6:     for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
7:        $A[n_1][n_2] \leftarrow x[n_1 \cdot N_2 + n_2]$ 
                                     # Step 2:  $N_2$  independent column DFTs of size  $N_1$ 

8:   for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
9:      $col \leftarrow [A[0][n_2], A[1][n_2], \dots, A[N_1-1][n_2]]$ 
10:     $col\_dft \leftarrow \text{DFT}(col, N_1)$ 
11:    for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
12:       $A[k_1][n_2] \leftarrow col\_dft[k_1]$ 
                                     # Step 3: Pointwise twiddle factor multiplication evaluated on the fly

13:  for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
14:    for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
15:       $W \leftarrow \exp(-i \frac{2\pi}{N} \cdot k_1 \cdot n_2)$                                      # Dynamic calculation
16:       $A[k_1][n_2] \leftarrow A[k_1][n_2] \cdot W$ 
                                     # Step 4:  $N_1$  independent row DFTs of size  $N_2$ 

17:  for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
18:     $row \leftarrow [A[k_1][0], A[k_1][1], \dots, A[k_1][N_2-1]]$ 
19:     $row\_dft \leftarrow \text{DFT}(row, N_2)$ 
20:    for  $k_2 \leftarrow 0$  to  $N_2 - 1$  do
21:       $B[k_1][k_2] \leftarrow row\_dft[k_2]$ 
                                     # Step 5: Read out transposed matrix into 1D output

22:  for  $k_2 \leftarrow 0$  to  $N_2 - 1$  do
23:    for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
24:       $X[k_2 \cdot N_1 + k_1] \leftarrow B[k_1][k_2]$ 
25:  return  $X$ 

```

3 Implementation

4 The Empirical Study

5 Lesson learned

6 AI usage Declaration